阶段一：密钥批量审计与 Asmuth-Bloom 门限重组

漏洞分析

题目提供了 20 个被用于加密 Asmuth-Bloom 门限方案碎片的 RSA 公钥。经过初步侦察，这批密钥的生成质量极差，存在以下四类典型缺陷：

1. 模数极小：可以直接通过外部数据库（FactorDB）或本地试除分解。
2. 随机数熵池重复：不同密钥的模数之间存在公因数（GCD 攻击）。
3. 素数间距过小：导致可以被 Fermat 分解法快速击破。
4. 参数错位：生成密钥对时，误将私钥指数 d 当作公钥 e 暴露（此时真正的 e=65537 实际上就是解密密钥）。

拿到足够的私钥后，我们解密出各个 AES-GCM 密钥及对应的文件份额 (mi,ri)。由于 Asmuth-Bloom 方案的核心是基于模算术的重组，收集到足够份额后，利用中国剩余定理（CRT）即可提取被隐藏的信息。

自动化破译脚本

集成化的审计与 CRT 恢复脚本：

代码块

```
import os, glob, math
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Util.number import bytes_to_long, long_to_bytes
from functools import reduce


os.chdir("D:\桌面\1\level1")

def fermat_factor(n):
    a = math.isqrt(n) + 1
    b2 = a*a - n
    while True:
        b = math.isqrt(b2)
        if b*b == b2:
            return a - b, a + b
        b2 += 2*a + 1
        a += 1
```

```python

def solve_crt(remainders, moduli):
    total_prod = reduce(lambda x, y: x * y, moduli)
    result = 0
    for r, m in zip(remainders, moduli):
        p = total_prod // m
        inv = pow(p, -1, m)
        result = (result + r * inv * p) % total_prod
    return result

def exploit_level1():
    # 1. 密钥加载
    rsa_keys = {f: RSA.import_key(open(f).read()) for f in glob.glob("key-*.pem")}
    cracked_d = {}

    # 2. 漏洞批量打击
    # - FactorDB (已查表)
    p1, p2, p3 = 13734854446830088769, 15950473204537742369, 1528159911436197621287
    cracked_d["key-5.pem"] = pow(3, -1, (p1-1)*(p2-1)*(p3-1))

    # - 共享因子扫描
    for f1, k1 in rsa_keys.items():
        for f2, k2 in rsa_keys.items():
            if f1 != f2:
                g = math.gcd(k1.n, k2.n)
                if 1 < g < k1.n:
                    cracked_d[f1] = pow(65537, -1, (g - 1) * (k1.n // g - 1))

    # - 费马攻击 (近距离素数)
    for f in ["key-3.pem", "key-7.pem", "key-13.pem", "key-18.pem"]:
        p, q = fermat_factor(rsa_keys[f].n)
        cracked_d[f] = pow(65537, -1, (p - 1) * (q - 1))

    # - 公私钥参数混淆 (直接将65537作为解密指数)
    for f in ["key-6.pem", "key-12.pem", "key-17.pem"]:
        cracked_d[f] = 65537

    # 3. 混合解密引擎
    valid_shares = []
    for i in range(1, 11):
        ct = open(f"ciphertext-{i}.bin", "rb").read()
        for k_file, priv_d in cracked_d.items():
            pub = rsa_keys[k_file]
            k_len = (pub.n.bit_length() + 7) // 8
            if len(ct) <= k_len + 28: continue
```

```
64
65              enc_key, nonce, enc_msg, tag = ct[:k_len], ct[k_len:k_len+12],
    ct[k_len+12:-16], ct[-16:]
66
67              try:
68                  # 区分填充模式处理
69                  if pub.n.bit_length() >= 2048:
70                      curr_e = pub.e if priv_d != 65537 else 65537
71                      curr_d = priv_d if priv_d != 65537 else pub.e
72                      aes_key = PKCS1_OAEP.new(RSA.construct((pub.n, curr_e,
    curr_d))).decrypt(enc_key)
73                  else:
74                      curr_d = priv_d if priv_d != 65537 else 65537
75                      aes_key = long_to_bytes(pow(bytes_to_long(enc_key),
    curr_d, pub.n)).rjust(16, b'\x00')[-16:]
76
77                  cipher = AES.new(aes_key, AES.MODE_GCM, nonce=nonce)
78                  pt = cipher.decrypt_and_verify(enc_msg, tag).decode()
79
80                  # 提取碎片用于后续重构
81                  target_line = pt.strip().split("\n")[6] # 第7个消息的碎片
82                  m_hex, r_hex, _ = target_line.split(":")
83                  valid_shares.append((int(m_hex, 16), int(r_hex, 16)))
84                  break
85              except Exception:
86                  continue
87
88      # 4. CRT 提取最终 Flag
89      if len(valid_shares) >= 7:
90          moduli = [s[0] for s in valid_shares[:7]]
91          rems = [s[1] for s in valid_shares[:7]]
92          secret_int = solve_crt(rems, moduli)
93          print("Level 1 ->", long_to_bytes(secret_int).decode())
94
95  exploit_level1()
```

阶段二：拓展版 Wiener's Attack

分析

该关卡的 RSA 参数 d 虽然相对较小（符合维纳攻击的界限要求），但使用的是 Carmichael 函数定义 $e \cdot d \equiv 1 \pmod{\text{lcm}(p-1, q-1)}$。这就导致了标准的求 $\phi(N)$ 的维纳脚本会失效。 对此，我们可以利用连分数展开 e/N 获取收敛常数对 (K,D)，并在遍历时引入松弛变量 c 和 g 修正同余方程，当判别式算出完美平方数时，说明推算出了正确的 p+q。

拓展攻击脚本

代码块

```
import math, hashlib

def get_convergents(e, n):
    x, y = e, n
    n1, d1, n2, d2 = 1, 0, x//y, 1
    yield n2, d2
    while x % y != 0:
        x, y = y, x % y
        a = x // y
        n1, d1, n2, d2 = n2, d2, a*n2 + n1, a*d2 + d1
        yield n2, d2

def solve_level2():
    n = 995733630482752347642314027694641164160870100149923192212010939056874399336324304660679920370461207121995652504821970043013433419606553579445773308854709184660077305707186480251435616563957515184286307425870232674506338246369369535248687352636660894523484660181950994715358239693650071206805465929990221957815
    e = 120768305392951935330332122324875688882009631230241892876294934800586382221469724961108143728838297656926231071911293061907889767042505023162654399968917641014470171903770149802935897974030952495383915349866389730352859008675484201922112411637789190289215023057909798803460504288391028740860466228332112119132991
    for K, D in get_convergents(e, n):
        if K == 0: continue
        for c in range(1, 100):
            for g in range(2, 500, 2):
                numerator = e * c * D - g
                denominator = c * K
                if numerator % denominator == 0:
                    L_cand = numerator // denominator
                    sum_pq = n - L_cand + 1
                    delta = sum_pq**2 - 4*n
                    if delta >= 0 and math.isqrt(delta)**2 == delta:
                        h = hashlib.sha256(str(sum_pq).encode()).hexdigest()
                        print("Level 2 Hash ->", h)
                        return

solve_level2()
```

得到解压校验 Hash：
`2aa9c360df99cbb4209e4dbab5a9f9ffd86d34906e3206fecfdabf0bb7aeb5ac`。

阶段三：基于按位约束扩展（LSB Oracle）还原大素数

分析

最后这道题给出了 3072 位 N 的长串复杂位运算混淆结果 `leak`。由于在计算机加法和按位异或运算中，第 k 位的进位或结果只取决于第 0...k 位。 基于这一特性，我们可以采用从低到高逐位测试（Bit-lifting）的策略： 设已正确推导出 p 的低 k−1 位，当猜想第 k 位时，可以通过模反向 $q \equiv N \cdot p^{-1} \pmod{2k}$ 唯一确定 q 的低 k 位。将其代入泄露多项式核对，一致则入队，否则剪枝抛弃。

注：Python 3.8+ 原生支持 `pow()` 的三参数模逆，这极大简化了计算特定截断位数下的 q 的过程。

按位搜索恢复脚本

代码块

```python
from Crypto.Util.number import long_to_bytes
import sys

# (此处省略 N, c, leak_val 等硬编码长常数，请填入题目原始数据)
# CONST1, CONST2, CONST3 等常数一致

def simulate_lsb(p_tmp, q_tmp, bit_len):
    m = (1 << bit_len) - 1
    t1 = (p_tmp * CONST1) & m
    t2 = (q_tmp * CONST2) & m
    t3 = ((p_tmp & q_tmp) << 64) & m
    t4 = ((p_tmp | q_tmp) << 48) & m
    t5 = ((p_tmp ^ q_tmp) * CONST3) & m

    val1 = t1 ^ t2 ^ t3 ^ t4 ^ t5
    val2 = ((p_tmp + q_tmp) % MOD128) & m
    val3 = (n & MASK64) & m

    return ((val1 + val2) ^ val3) & m

def solve_level3():
    possible_p = {1} # LSB必须是1（奇数）

    for current_bit in range(1, 1536 + 1):
        mask = (1 << current_bit) - 1
        target_leak = leak_val & mask
        new_possible = set()
```

```
28
29          for p_cand in possible_p:
30              for bit_val in (0, 1):
31                  test_p = p_cand | (bit_val << (current_bit - 1))
32                  try:
33                      # 利用原生 pow 快速计算 mod 2^k 域内的逆元
34                      test_q = (n * pow(test_p, -1, 1 << current_bit)) & mask
35                  except ValueError:
36                      continue
37
38                  if simulate_lsb(test_p, test_q, current_bit) == target_leak:
39                      new_possible.add(test_p)
40
41          possible_p = new_possible
42          if not possible_p:
43              print(f"Error: Path failed at bit {current_bit}")
44              return
45
46      # 从候选集合中验算 N
47      for p in possible_p:
48          if n % p == 0:
49              q = n // p
50              d = pow(65537, -1, (p-1)*(q-1))
51              print("Level 3 Flag ->", long_to_bytes(pow(c, d, n)).decode())
52
53  solve_level3()
```

运行该剪枝算法，飞速完成 1536 次迭代，顺利输出最终 Flag：
`dart{379c9308-e9a8-45a1-bd55-45bbd822e86d}`